

Relic Build Optimizer

Mathematical Formulation, Guarantees, and Empirical Validation

zestones · github.com/zestones/Evil-Nightreign

July 2026

This document formalizes the relic build optimization problem, proves the structural properties the algorithm relies on, and validates them on the real data under `data/curated/`. Companion to [optimizer.md](#). Every data-level assumption used by a proof is machine-checked by `experiments/validate_invariants.py`; re-run it whenever the curated data is regenerated.

Contents

1. Notation and problem	4
2. Effect aggregation (the stacking rules)	5
2.1 Action-gated offense (the play profile)	6
2.2 Status ailments	7
3. Dominance pruning	10
4. Search and guarantees	12
5. Empirical validation (real data)	13
5.0 Data invariants	13
5.1 Proposition 1 — submodular, and strictly so	13
5.2 Theorem 2 — pruning is lossless (not just smaller)	14
5.3 Greedy vs beam — where greedy actually fails	14
6. Consequences for the implementation	16

1. Notation and problem

Fix a **context** $x = (\text{character}, \text{weapon type}, \text{affinity}, \text{toggles}, \text{target}, \text{DoN level}, \text{party})$. Everything below is relative to x ; the outer loop iterates over the free parts of x .

- R — the owned relics. Each $r \in R$ has a color $\text{col}(r) \in \{R, B, Y, G\}$, a type $\text{typ}(r) \in \{\text{normal}, \text{deep}\}$ (unique relics behave as normal-pool relics), and up to 3 effect instances $E(r)$.
- Each effect instance e has a **key** $k(e)$ — its identity in the effect table — a group $g(e)$ (or $\perp$), a level $\ell(e) \in \mathbb{N}$, a stack flag $\sigma(e) \in \{0, 1\}$, a magnitude, and a gate $\text{cond}(e)$. Effect e is **active** in context x iff $\text{cond}(e)$ is satisfied by x .
- A **vessel** V gives colored slots: three normal V_n and three deep V_d ; a slot's color is one of $\{R, B, Y, G\}$ or **Any**.

Two data invariants pin down the right unit of aggregation (both machine-checked; 485 keys, 54 groups, 1823 relics):

- **(INV-1)** σ is a function of the **key** — no key mixes flags across its copies. It is **not** a function of the group: 3 of the 54 groups mix flags across their levels (e.g. `improvedCriticalHits`: level 0 stacks, level 1 does not). Any formalization of stacking “per group” is therefore wrong on real data.
- **(INV-2)** Within a group, $k \leftrightarrow (g, \ell)$ is a bijection carrying a single magnitude (140 pairs), so “the effect of group g at level ℓ ” is well defined and *is* a key.

Feasible sets form a matroid. A **build** is an injective placement φ of relics into slots with $\text{col}(\varphi(s)) = \text{col}(s)$ or $\text{col}(s) = \text{Any}$, and typ matching the slot's pool; write $A = \text{im}(\varphi) \subseteq R$ for the equipped set. The placeable sets A are exactly the independent sets of the transversal matroid $\mathcal{M}$ of the bipartite relic-slot color-compatibility graph. Because the normal and deep pools use disjoint relics and disjoint slots,

$$\mathcal{M} = \mathcal{M}_{\text{normal}} \oplus \mathcal{M}_{\text{deep}}$$

is a direct sum of two transversal matroids (rank 3 each) — still a matroid, of rank ≤ 6 .

Objective.

$$S(A) = w \cdot \text{OFF}(A) + (1 - w) \cdot \text{SURV}(A),$$

with the two axes built from the *aggregated effect state* below. (Effects do not depend on which slot holds the relic, so S is a function of the set A alone.)

2. Effect aggregation (the stacking rules)

For a per-type magnitude field, let $v(e)$ be the effect’s contribution in **log space** ($v = \log(\text{multiplier})$ for a multiplicative buff, 0 if it does not touch this field).

- **(INV-3, verified $v \geq 0$ on non-debuff effects)** On the owned collection no *buff* lowers a tracked multiplicative field: there is no `*AttackRate` < 1 and no `*DamageCutRate` > 1 on any non-debuff relic effect, deep included. Nonnegativity is a **verified data invariant, asserted at load time** for non-debuff keys. Deep-of-Night **curses** deliberately break it — they *are* the malus (`*AttackRate` < 1 , `*DamageCutRate` > 1 , `maxHpRate` < 1 , `add*Status` < 0). Since each curse is inseparable from its buff (same relic) and the log representation is sign-agnostic, they enter as **signed** contributions $v(e)$ that may be < 0 : the only load-bearing guard is the log domain $v > 0$ (multiplier), the sign is carried by the monotone exp downstream. Consequence: monotonicity (Prop. 1) is lost on cursed pools — adding a cursed relic can lower an axis — so Agg’s codomain is $\mathbb{R}$, not $\mathbb{R}_{\geq 0}$. The greedy $\frac{1}{2}$ -bound (§4) no longer holds there, but the search already rests on **beam + exhaustive verification** (§5.3), and **dominates** iterates the **key union** so a cursed dominator cannot falsely prune a clean relic (Theorem 2 stays sound).

Aggregation is **per key**. Writing A_k for the active copies of key k carried by A (a relic may carry several copies of one key — one real case),

$$\text{Agg}(A) = \underbrace{\sum_{k: \sigma(k)=1} \sum_{e \in A_k} v(e)}_{\text{stacks with itself: every copy adds}} + \underbrace{\sum_{k: \sigma(k)=0, A_k \neq \emptyset} \max_{e \in A_k} v(e)}_{\text{one instance counts}}$$

Consequences of the invariants:

- For a grouped $\sigma = 0$ key this is exactly “one instance per distinct (g, ℓ) ”: distinct levels are distinct keys (INV-2) and **coexist — verified in game** (§5.0): a relic carrying both levels of `improvedCriticalHits` multiplies critical damage by $\times 1.452 \approx 1.18 \times 1.23$, not by $\max(1.18, 1.23)$.
- For $\sigma = 1$ keys the group is irrelevant: **all copies add, including ungrouped stacking keys** — 102 real keys, 996 occurrences (e.g. `fireDamageNegationUp`, `improvedInitialStandardAttack`). An earlier draft capped ungrouped keys at one instance, which under-counts all of these; and it keyed σ off the group, which is ill-defined on the 3 mixed groups (INV-1).
- The max makes Agg a well-defined *set function* even for the three keys whose copies carry different magnitudes (`increasedMaximumFP`, `increasedMaximumHP`, `takingAttacksImprovesAttackPower`); a “first/last copy wins” rule would be order-dependent.

Proposition 1 (aggregation is monotone submodular). $\text{Agg} : 2^R \rightarrow \mathbb{R}_{\geq 0}$ is monotone and submodular.

Proof. $\text{Agg} = \sum_k f_k$, and a sum of submodular functions is submodular, so it suffices to check each per-key term.

- $\sigma(k) = 1$: $f_k(A) = \sum_{e \in A_k} v(e)$ is **modular** (the marginal of any relic is constant), hence submodular.
- $\sigma(k) = 0$: $f_k(A) = \max_{e \in A_k} v(e)$ (with $\max_{\emptyset} = 0$) is a rank-one **facility-location** term — adding a relic can only raise the max, and its increment against a larger set is no larger.

Each $v(e) \geq 0$ (INV-3) gives monotonicity. ■

Offense. For the dominant damage type τ (set by the affinity), $\text{OFF}(A) = \text{base}_\tau \cdot \exp(\text{Agg}_\tau(A))$, i.e. $\text{base} \times \prod$ multipliers, under two stated provisos:

1. base_τ is treated as **constant in A**: stat-scaling effects ($+\text{Str} \Rightarrow$ higher AR) are excluded from this analysis and handled at search time (§6.5). *Multi-source extension (2026-07-17)*: each damage SOURCE (weapon attack, spell cast, character skill/ultimate — `engine/sources.py`) contributes its **own constant base** base_s ; the relic side stays inside the same per-key Agg, so nothing below changes. The catalyst stat-feedback ($+\text{Int} \Rightarrow$ higher spell base) is a **second instance of the §6.5 coupling**, handled by the same memoized fixed point (`Scorer._source_power` shares `_ar`’s cache key) — no new proof obligation.
2. This is **per damage type**. If the implementation scores the full split AR (physical + elemental), OFF becomes a *sum* of exponentials across types and leaves the submodular regime exactly like the two-axis objective (§4); the guarantees here are per-type.

Since $\exp$ is strictly increasing,

$$\arg \max_A \text{OFF}(A) = \arg \max_A \text{Agg}_\tau(A).$$

The multiplicative objective and the submodular log-objective have the **same optimizer** — the log transform preserves the argmax without changing the problem. It preserves *only* the argmax: approximation ratios do **not** transfer through $\exp$ (§4).

Survival. The effective-HP multiplier is $\exp(-\sum \log(\text{cutRate}))$, again $\exp(\text{submodular})$ (cut rates ≤ 1 on the data, INV-3); the same argument applies per axis.

2.1 Action-gated offense (the play profile)

Of the 123 keys touching an `*AttackRate` field, only 59 apply to every attack; the other 64 are **gated to a specific action** by three engine mechanisms decoded from the raw `SpEffect` rows (`resources/actions.py`, extracted into `effects.json` as `actions / state_gate`):

1. `magicSubCategoryChange1/2/3` — attack sub-categories, self-labeled by the keys that carry them: melee (130), first standard attack (119), skill (111/112), guard counter (103), throwing knife (120), pots (108), glintstone/gravity stones (121), perfumes (109), roars (106/107/116), chain finisher (104), and 14 sorcery/incantation schools (2–26).
2. `stateInfo` — hardcoded behaviors: 367 = critical hits only (**game-verified**: the R1 control stayed at 125 in every crit-relic run of §5.0 while backstabs scaled), 2100 = “3+ armaments of the type equipped” (a *loadout state*, exposed as the `triple_loadout` toggle).

3. `magParamChange` / `miracleParamChange` — generic offense carries **both** flags; exactly one flag (and no finer gate) marks the six generic spell buffs (`improvedSorceries*` → all sorcery schools, `improvedIncantations*` → all incantation schools). Verified exhaustively: on the owned collection the one-flag keys are *exactly* those six plus the school-gated ones.

Model. Each key k gets an action class $\alpha(k) \in A \cup \{*\}$; the player declares a **play profile** $p = (p_a)_a$, $\sum_a p_a = 1$ — the share of their damage each action carries (default: the pure-melee benchmark $p = \{\text{melee} : 1\}$). Writing $C(a)$ for the classes applying to action a (*, a itself, and the spell umbrella of a 's school),

$$\text{OFF}(R) = \sum_a p_a \cdot D\left(\text{base}_{s(a)} \cdot \exp\left(\sum_{c \in C(a)} \text{Agg}_c(R)\right)\right),$$

with D the damage pipeline and $\text{base}_{s(a)}$ the attack-power base of action a 's SOURCE — the equipped weapon's $\text{AR} \times \text{MV}(a)$ for weapon-family actions, the resolved spell/skill base for sourced actions (`engine/sources.py`; a declared cast with **no** resolved catalyst spell has $\text{base} = 0$, never a silent weapon-AR fallback). Three consequences. (i) Per action, the objective is still $D(\text{base} \cdot e^{\text{submodular}})$; the mixture over actions — now a weighted sum of exponentials with **per-source bases** — stays monotone, not submodular: exactly the regime §4 already assigns to beam + exhaustive verification, no new machinery (re-verified empirically post-extension: $\text{beam}(k=12) = \text{exhaustive}$ on a mixed sorcery/melee Recluse context, `tests/test_sources.py` companion check). (ii) Theorem 2 is untouched: profile dimensions refine from (atk, type) to (atk, type, class) and the proof never counts dimensions — **Proposition 1 and Theorem 2 hold unchanged as long as the relic aggregation stays per-key log-additive**, which no source touches. (iii) Under the default profile a crit-only or throwing-knife-only relic is worth exactly nothing — matching the measured game behavior instead of the $\times 8.5$ fantasy multipliers an ungated model produces.

2.2 Status ailments

Every hit adds the weapon's buildup (`SpEffect bloodAttackPower`, ... — chain validated on Reduvia: 45/hit) plus the relic on-hit buildup (aggregated per key like everything else, dimension (stbuild, s)) to the target's gauge, which procs at its resistance threshold.

The threshold escalates after every proc (`ResistCorrectParam`, per boss, extracted into `nightlords.json`). Per the game's own field annotations the gauge *resets* after a proc, and after N activations the fill target becomes $\text{base} \cdot \text{addRate}_N + \text{addPoint}_N$; the cumulative buildup to the N -th proc is the running sum of those per-proc fills. For Gladius bleed (base 252, `addPoint` 290, 747, ...) the cumulative thresholds are 252, 794, 1793, ..., i.e. the procs land at $\sim 5.6, 17.6, 39.8$ hits (at 45/hit). This is derived from the field definitions, and *independently* matches the player's rough "first proc ~ 5 hits, second ~ 20 ". So bleed is *front-loaded*: a full fight yields only ~ 3 procs, not one every 5.6 hits. A steady buildup/resistance rate overcounts it $\sim 3\times$ and wrongly ranked bleed daggers over the holy Black Knife vs the holy-weak Gladius; the escalation model corrects that (holy now wins, matching play experience).

Expected status damage is therefore amortized over the fight: with $H = \text{HP}/\text{direct-dmg-per-hit}$ the estimated hit count, $n_s = \#\{k : \text{threshold}_k \leq H \cdot \text{buildup}_s\}$ procs land, contributing per hit

$$\text{dmg/hit} += \sum_s \frac{n_s \cdot \text{proc}_s(\text{HP}_{\text{target}})}{H}$$

Two proc shapes, both escalation-aware:

- **Burst** (bleed, frost): each proc is a one-shot % of max HP, so fight damage = $n_s \cdot \text{burst}_s$. **bleed** = 15% — *locked* by the player-measured 371 proc (day-3 troll) and the engine’s canonical hemorrhage row (SpEffect 1612); **frost** = 5% (SpEffect 1611, structural twin — to confirm).
- **DoT** (poison, rot): a proc applies a damage-over-time that later procs *refresh* rather than stack, so once the first proc lands the DoT is up until its last application’s duration lapses or the fight ends. Fight damage = $\text{uptime}_s \cdot \text{rate}_s$, with $\text{uptime}_s = \min(\text{fight}_s, \text{last-proc}_s + \text{dur}_s) - \text{first-proc}_s$ and hits $\leftrightarrow$ seconds via the weapon’s class cadence. This replaces an earlier per-proc $\times 0.5$ approximation that *double-counted* overlapping DoTs (poison on a boss: physically $\leq \text{rate} \times \text{fight}$, but the old model summed a half-DoT per proc). **poison** = 40s at 1.3s ticks, 10 + 0.10% HP/tick; **rot** = 40s, 15 + 0.15% HP/tick (structural, from the stateInfo-2/-5 SpEffect rows).

Resistance 999 = immune. Sleep/madness/curse carry no damage term (control/niche). Deep-of-Night HP scaling applies to the proc values. Frost additionally applies **damage taken** $\times 1.15$ **for** 30s (a 195-row SpEffect family, stateInfo 260) — near-permanent uptime once procced, so a frost weapon multiplies the whole build’s damage against non-immune targets. Only bleed’s proc value is player-validated; frost/poison/rot values are structural (from the params).

***D* is linear (game-verified 2026-07-15).** Every NPC carries the same inert flat defense (100 on all 200 rows), and a two-dagger duel on such an enemy dealt *exactly* the displayed attack values (103 and 122) where the Elden Ring attack/defense curve predicts a $\times 0.40$ multiplier: Nightreign player damage is $\text{attack} \times \text{MV} \times \text{cutRate}$, with the target’s per-element multipliers as the only defense-side term. An earlier draft routed damage through the ER curve, whose per-type compression silently punished split-damage (elemental) weapons — with the linear *D*, elemental weapons against a matching weakness get their full face value.

In-game verification of the action gates (2026-07-15; Wylder, training grounds, same target; the floating damage number accumulates rapid consecutive hits, and the R1 chain resets when pausing — slow-paced hits are all *first* hits):

equipped	1st chain hit	chained 2nd hit	verdict
none	125	126	baseline
melee $\times 1.06$ (sub-cat 130)	131	133	whole chain buffed ($\times 1.05$, display-rounding of 1.06)
first-hit $\times 1.15$ (sub-cat 119)	144	126	first hit only ($\times 1.152$; the chained hit is exactly baseline)

Both gates behave as decoded, and the first chain hit received **both** buffs across configs — an initial standard attack *is* a melee attack. A second session (weapon with a damaging art, carrier of melee $\times 1.06$ + skill $\times 1.15$) settled the hierarchy:

measure	without	with	ratio	verdict
R1 (control)	142	149	$\times 1.049$	melee buff, same display damping as above
weapon art (L2)	152	182	$\times 1.197$	$\approx 1.15 \cdot 1.06$ — skills inherit melee buffs (skill alone predicts ~ 171 – 175)
backstab (crit)	286	300	$\times 1.049$	exactly the melee ratio — crits inherit melee buffs, and not skill buffs

The class hierarchy therefore treats **initial**, **skill**, **crit** (and, by extrapolation, **guard_counter**, **chain_finisher** — same melee-performed family, unmeasured) as subclasses of **melee**. Practical corollary: at a hit-and-run pace the chain keeps resetting, so first-hit buffs apply to almost every hit — exactly what a play profile like **melee=0.6**, **initial=0.4** expresses. The crit gate (367) itself was verified earlier (§5.0).

3. Dominance pruning

Definition (profile, dominance). In context x , a relic’s **profile** assigns to every key k of every axis: the *sum* of its active copies’ values if $\sigma(k) = 1$ (intra-relic copies exist — sum, don’t max), the max otherwise. Relic r' **dominates** r if col, typ are equal and $\text{profile}(r') \geq \text{profile}(r)$ on every key, strictly somewhere.

The naive rule — *one dominator suffices to discard* — is **false** as soon as two slots accept the same color:

Counterexample (single-dominator pruning is lossy). One $\sigma = 1$ key g ; all relics the same color; slots [Any, Any, ·]. Let r' carry g at 0.20, r carry g at 0.10, z carry junk at 0.01. Then r' strictly dominates r , so r is discarded — but $\text{OPT}_{\text{full}} = \{r', r\} = 0.30$ (the copies stack) while $\text{OPT}_{\text{pruned}} = \{r', z\} = 0.21$: a 30% loss. The flaw in the naive exchange proof is the case “ r' already equipped”: removing r then forfeits its stacking contribution outright. The mechanism is present in the real collection — 33 same-color (dominant, dominated) pairs share an active $\sigma = 1$ group in the fire context alone.

The multiplicity-aware rule is safe:

Theorem 2 (safe pruning). Fix the vessel and pool. For a color c let s_c be the number of slots that accept c (an Any slot accepts every color; $1 \leq s_c \leq 3$). If at least $s_{\text{col}(r)}$ distinct relics strictly dominate r , then some optimal build omits r ; removing **all** such relics simultaneously is safe.

Proof. Among optimal builds choose A^* using the fewest discarded relics, and suppose a discarded $r \in A^*$, with dominator set D , $|D| \geq s_c$. All of $D \cup \{r\}$ share r ’s color and type, so they can only occupy the s_c compatible slots; r holds one, hence at most $s_c - 1$ members of D are equipped and some $d \in D \setminus A^*$ exists. $A' = A^* - r + d$ is feasible (d takes r ’s slot). On every axis $\text{Agg}(A') \geq \text{Agg}(A^*)$: on $\sigma = 1$ keys the aggregation is modular and $\text{profile}(d) \geq \text{profile}(r)$ pointwise; on $\sigma = 0$ keys every key value r contributed is matched or beaten by d ’s, so no per-key max drops. S is increasing in each axis, so A' is optimal too. Each such swap replaces a relic by one strictly above it in the (finite, acyclic) strict-dominance order, so iterating terminates in an optimal build avoiding every discarded relic. ■

Corollary. On a vessel whose slots have pairwise-distinct colors and no Any slot ($s_c = 1$ everywhere), a single strict dominator suffices — the regime of §5.2. As soon as several slots accept one color the count matters: real vessels have repeated colors (Wylder’s Urn $[R, R, B]$) and the mono-color Grails (Giant’s Cradle $[B, B, B], \dots$) realize the worst case $s_c = 3$. Any slots are rarer than the params suggest: the Chalices — the only vessels with a normal-pool Any slot — are fully wired in the game data but currently **unobtainable** (verified against the save: no Chalice owned with every bazaar purchase done), so on the owned collection the only Any slots are the Forgotten Goblets’ third *deep* slot. `vessels.json` carries an **owned** flag (save-derived) and the optimizer must restrict to **owned: true**. §5.3 prunes with the s_c -aware rule. (Slot colors verified in game against Wylder’s full vessel list — `vessels.py` documents the color enum.)

Pruning must be **context-aware**: cond changes which effects are active, so a relic dominated under one weapon may be unique under another. Pruning therefore runs inside the outer loop.

Implementation note. Dominance comparisons use a numerical tolerance ε ; relics whose profiles are ε -equal must be treated as ties (kept, or deduplicated by an explicit tie-break) — otherwise two ε -equal relics can each count as the other’s “strict” dominator and both be discarded.

4. Search and guarantees

Single axis. Maximizing a monotone submodular function over a matroid, the **matroid greedy** — at each step add the feasible element with the best marginal — is a $\frac{1}{2}$ -approximation (Fisher–Nemhauser–Wolsey, 1978), and continuous greedy (Calinescu–Chekuri–Pál–Vondrák) achieves $1 - 1/e$. Two precisions an earlier draft missed:

- The bound lives in **log space**: $\text{Agg}(\text{greedy}) \geq \frac{1}{2} \text{Agg}(\text{OPT})$, which exponentiates to $\text{OFF}_{\text{greedy}} \geq \sqrt{\text{base}_\tau \cdot \text{OFF}_{\text{OPT}}}$ — a geometric-mean bound, **not** “half the multiplier”. Only the argmax survives exp; ratios do not.
- The **slot-ordered greedy** actually implemented (fill slot 1, then slot 2, ...) is *not* the matroid greedy and inherits no bound: an Any slot processed before a scarce color can absorb that color’s only good relic — a failure mode that bites even *modular* objectives.

Two axes. $S = w e^{\text{Agg}_{\text{off}}} + (1 - w) e^{\text{Agg}_{\text{def}}}$ is monotone but, being a weighted sum of two exponentials, **not submodular in general**. For monotone objectives one bounds the *submodularity ratio* $\gamma \in (0, 1]$; greedy then yields $\geq (1 - e^{-\gamma})$ of the optimum (cardinality case) or a matroid analogue. We do not rely on a worst-case γ : after pruning the instance is small, so we **verify against the exhaustive optimum** and use **beam search** (keep the top- k partial builds). Section 5.3 measures where slot-ordered greedy actually leaves the optimum — the observed failures turn out to be matroid/ordering phenomena, not two-axis ones — and shows beam recovering it in every case; beam is load-bearing either way.

5. Empirical validation (real data)

Each claim is tested by a script under `experiments/` on the owned collection. The design principle here is that a test must be able to *fail*: a validation that only ever lands in the easy regime confirms nothing. So each test is run where its guarantee is actually at risk. All scripts use the per-key aggregation of §2; the earlier per-group draft produced the *same numbers* on every instance below (re-verified), but that is a property of this collection, not of the model — the per-group rule is order-dependent on the 3 mixed groups and under-counts the 102 stacking ungrouped keys. Numbers below are the collection snapshot of **2026-07-15** (1823 relics); re-run the scripts after every `nr data` refresh (a refresh from 1632 to 1823 relics left §5.1 and §5.3 unchanged and only moved the §5.2 pool counts).

5.0 Data invariants

Script: `experiments/validate_invariants.py` — asserts INV-1/2/3 (all **PASS** on the current data: 485 keys, 140 (g, ℓ) pairs, 0 negative contributions over 5 attack + 9 cut fields, 0 unresolved effect ids out of 5335) and reports the known quirks the model must absorb: 3 keys with several magnitudes across their ids, and 1 relic carrying the same $\sigma = 1$ key three times (which is why $\sigma = 1$ profile entries *sum* intra-relic copies).

In-game verification of the stacking rules (2026-07-15; Wylder, level 1, starting weapon, backstab on the same base soldier; baseline crit 241, normal-hit control 125 unchanged across every run — the crit multipliers touch nothing else):

equipped	crit	ratio vs baseline	verdict
carrier of <i>both</i> levels of <code>improvedCriticalHits</code>	350	$\times 1.452 \approx 1.18 \cdot 1.23$	levels coexist
+ a second copy of the $\sigma = 0$ level ($\times 1.23$)	350	unchanged	$\sigma = 0$: one instance
one copy of the $\sigma = 1$ level ($\times 1.18$)	282	$\times 1.170$	calibration
two copies of the $\sigma = 1$ level	335	$\times 1.188$ vs the single copy	$\sigma = 1$: copies stack

Measured ratios match the extracted magnitudes within $\sim 1\%$ (the residual is the damage pipeline’s defense curve, not stacking — the discriminating gaps were 20%+). All three aggregation rules of §2 are now game-verified, and the extracted magnitudes (1.18, 1.23) are confirmed against real damage numbers.

5.1 Proposition 1 — submodular, and strictly so

Script: `experiments/validate_submodularity.py`. We isolate a single per-key term f_k (the proof decomposes $\text{Agg} = \sum_k f_k$) carried by three real relics at the **same** (g, ℓ) — i.e. the same key — and read its marginals:

key (= group @ level)	σ	1st	2nd	3rd	regime
attackPowerUp...ScarletRo0...@00	0.0953	0.0000	0.0000	0.0000	strict diminishing (coverage)
improvedThrowingKnifeDamage@0	0.1398	0.1398	0.1398	0.1398	constant (modular)

The $\sigma = 0$ row is the point: once the key is present, a second copy adds **zero** — a genuine strictly-submodular drop, not the trivial modular case. The $\sigma = 1$ row confirms the stacking keys are modular (the special case). Both are submodular, as claimed.

5.2 Theorem 2 — pruning is lossless (not just smaller)

Script: `experiments/validate_pruning.py`. Instance: **Duchess, Greatsword + Fire, vs a fire target**, colored slots $[R, B, G]$ — one slot per color, so $s_c = 1$ and Theorem 2 licenses the single-dominator rule. The real test is not that counts shrink but that the optimum survives, so we enumerate **both** pools exhaustively and compare optima (the ground-truth OPT is computed on the *full* pool precisely so it cannot trust the pruning it checks):

pool	per-color	combinations	OPT
full	$45 \times 40 \times 38$	68 400	0.7779800492
pruned	$34 \times 34 \times 33$	38 148	0.7779800492

$\text{OPT}_{\text{full}} = \text{OPT}_{\text{pruned}}$ to machine precision, while pruning discards 44.2% of the search space. (The counts move with each collection refresh — losslessness and, so far, the optimal value itself have been stable across refreshes.)

In the Any-slot regime the s_c -aware rule is the one that carries the guarantee; `validate_two_axis.py --verify-pruning` re-enumerates the *unpruned* pools across the full §5.3 sweep and finds $\text{OPT}(\text{pruned}) = \text{OPT}(\text{full})$ in all 60 contexts.

5.3 Greedy vs beam — where greedy actually fails

Script: `experiments/validate_two_axis.py`, objective $S = w e^{\text{Agg}_{\text{off}}} + (1 - w) e^{\text{Agg}_{\text{def}}}$, pools pruned with the s_c -aware rule of Theorem 2. The two swept vessels are **synthetic stress instances** (named after their slot lists): on the owned collection the only Any slots are the Forgotten Goblets’ single *deep* slot (§3), so a two-Any vessel over-couples relative to the game — deliberately, to put greedy at maximal risk. The real counterparts of the coupled regime are the repeated-color vessels and the mono-color Grails (s_c up to 3).

On the single-color instance $[R, B, G]$ per-slot greedy is *not* structurally exact, for two reasons. First, cross-color coverage coupling exists: 3 $\sigma = 0$ one-instance keys are active on more than one of the three colors in the §5.2 context (`improvedCriticalHitsPlus1` on both Blue and Red, `attackPowerUpAfterDefeatingANightInvader`, `attackPowerPermanentlyIncreased...`); the other 14 cross-color keys are $\sigma = 1$, i.e. modular and harmless. Second, for $0 < w < 1$ the myopic per-slot offense/defense trade-off can misfire even with fully separable Aggs (S is nonlinear in the two sums). Greedy merely *happens* to reach OPT in all 30 single-color swept

cases on this collection — an earlier draft called that regime “trivially exact”; the exactness is measured, not structural. The regime that actually separates the algorithms is a vessel with “**Any**” slots, where placement becomes a genuine bipartite matching over the transversal matroid.

Sweeping 60 contexts (3 elements $\times$ 2 toggle sets — none vs $\{\text{situational}, \text{low_hp}\}$, which gates 41 real effects — $\times$ 2 vessels $\times w \in \{0, 0.3, 0.5, 0.7, 1.0\}$; an earlier draft’s toggle sets matched condition *labels* that exist nowhere in the data, silently making that axis inert — toggles are **dimensions** of the condition taxonomy):

metric	greedy, vessel order	greedy, restrictive-first	beam ($k = 12$)
min / OPT	0.9134	0.9390	1.0000
mean / OPT	0.9947	0.9913	1.0000
cases < OPT	24/60	12/60	0/60

No static slot order is safe, and the failures come from two distinct mechanisms, always deepest at $w = 1.0$:

- *Vessel order* (Any slots filled first) fails on every toggle-less Urn [Any, Any, Red] case with $w \geq 0.3$ and, with toggles on, on both vessels — down to -8.7% (Urn, thunder, $w = 1.0$): the Any slots absorb relics a scarce colored slot needed — a **matching artifact** (it would bite even a modular objective).
- *Restrictive-first* ($[R, \text{Any}, \text{Any}]$, $[B, G, \text{Any}]$) repairs every Urn case but fails on the toggle-less Chalice [Any, Blue, Green] at $w \geq 0.3$ (up to -6.1%): the early Blue pick covers a $\sigma = 0$ key (`improvedCriticalHitsPlus1`) that the best Any-candidate also carries, collapsing its marginal — the genuinely **submodular coverage-overlap** trap.

Representative rows:

vessel	element	toggles	w	greedy / OPT	restr.-first / OPT	beam / OPT
Urn [Any, Any, Red]	fire	off	1.0	0.9907	1.0000	1.0000
Urn [Any, Any, Red]	thunder	on	1.0	0.9134	1.0000	1.0000
Chalice [Any, Blue, Green]	thunder	off	1.0	1.0000	0.9390	1.0000

Three reading notes. First, both mechanisms peak at $w = 1.0$, a **single-axis** weight where S is a monotone transform of the submodular Agg_{off} — so the measured shortfalls are *not* the two-axis sum-of-exponentials non-submodularity (an earlier draft misattributed them to it); that coupling remains a theoretical caveat this collection happens not to exhibit. Second, the gaps reach 8.7% and 6.1% depending on the order, and which order fails where depends on the toggle set — ordering heuristics carry no guarantee. Third, **beam (top- k , $k = 12$) reaches the exhaustive optimum in every one of the 60 cases**, and the s_c -aware pruning loses nothing against the full pools in all 60 (**-verify-pruning**) — this is the measured justification for beam: it is not a decorative safety net, it is load-bearing.

6. Consequences for the implementation

1. Work in **log space** for the multiplicative offense — it preserves the argmax (only the argmax: ratios don't transfer) and turns base $\times$ mult complementarity into an additive submodular objective, per damage type.
2. Implement the **per-key** Agg (§2): $\sigma(k)$ decides — copies add iff $\sigma(k) = 1$, else one instance (max over copies); never key stacking off the *group*. Assert INV-1/2/3 at data load (`validate_invariants.py` is the reference).
3. **Prune dominated relics** per context with the *s_c-aware rule* (Theorem 2); the single-dominator shortcut is only valid when every slot color is distinct and no slot is Any.
4. Search with **beam** ($k = 12$ measured sufficient; slot order then immaterial in the sweep). Slot-ordered greedy — under *any* static order — drops up to 1–6% below OPT on Any-slot vessels (§5.3); keep greedy only as a lower-bound warm start. Retain an exhaustive verifier on the pruned pool for regression tests, exactly as done here.
5. The only genuinely non-submodular coupling *inside one axis* — the **stat feedback** (+stat $\Rightarrow$ higher AR) — enters Agg_{off} 's base_τ and is handled by evaluating AR inside the marginal step (or one fixed-point pass). Split-AR weapons (physical + elemental) add a second such coupling if scored jointly (§2): both are reasons the exhaustive verifier stays in the loop.
6. Score offense through the **play profile** (§2.1): action-gated effects count only at the weight of the action they boost, evaluated through the damage pipeline per action. Never sum a gated multiplier into the generic offense — the game does not.